Credit Card Verification Application

This programming project is due on **Monday, November 17**.

Acknowledgement: This project assignment was adapted from one used by Professor Arland Richmond, who taught at BHCC for many years.

Table of Contents

1. Project Description 2
Credit Card Account Numbers and Issuer Codes 2
Data File Format 3
ALWAYS Update the "**dataFile.txt**" File 3
Processing 3
Transaction Types 3
Transaction Acceptance or Rejection 3
2. Initial Sample Input / Output 4
Resulting Data File 6
3. More Sample Input / Output 6
Final Data File 7
4. Design Suggestion: Two Classes 7
The "Credit" Class 7
The "CreditAccount" Class 8
5. Data File Archival Process 9
6. Project Deliverables: 9
7. Grading Criteria 9
Program Header Documentation 10
Names Matter 10
Neatness Counts 10
Project Grading Policy 10

1. Project Description

This programming assignment involves writing a text-based Java application that allows the user to CREATE and VERIFY credit card accounts. The user types textual commands that operate as shown in the examples below. (This sample list is provided to aid your understanding. It is *not* an exhaustive list of test cases for the program.)

`create v`

In this example, the user is requesting to CREATE a new "Visa" credit card account. The program must create a new account and assign it a credit limit as described later in this document.

`create AE`

In this example, the user is requesting to CREATE a new "American Express" credit card account. The program must create a new account and assign it a credit limit as described later in this document.

`verify 5439123412345678 250.00`

In this example, the user is requesting to process a **purchase** of $250.00.
First, the software must VERIFY that account 5439123412345678 exists and has at least $250.00 of available credit. If the account *does* have at least $250.00 of available credit, then the available credit is reduced by $250.00 and the program outputs a message indicating that the transaction was approved. This type of transaction is called a "debit".

`verify 5439123412345678 -125.00`

In this example, the user is requesting to process a **return** of $125.00.
First, the software must VERIFY that adding $125.00 to the available credit of account 5439123412345678 will not raise the available credit above the Maximum Credit Limit for this account. If the transaction is allowed, then the available credit is increased by $125.00, and the program outputs a message indicating that the transaction was approved. This type of transaction is called a "credit".

Credit Card Account Numbers and Issuer Codes

Each credit card account number must be represented in the program by a **String** object. The text we place in this **String** consists of 16 decimal digits: The first digit is assigned according to the table below. The remaining 15 digits must be generated randomly. When initially creating each account, the maximum credit limit is based on the last digit of the credit card account number. That is, if the last digit is in the range of **0-4**, then a maximum credit limit of **$1000.00** must be assigned. Otherwise, the maximum credit limit of **$500.00** must be assigned.

The following table lists several credit card issuers, the corresponding symbol that the user types in the "**create**" command, and the first digit of the account number for that particular issuer.

Card Issuer	Symbol	First Digit
American Express	AE	3
Visa	V	4
MasterCard	MC	5
Discover	DIS	6
Diners Club	DINE	7

Data File Format

The credit card data must be stored in a text file named "**dataFile.txt**". Each line of text in the file represents one credit card account, and the format of the data must be as follows:

account_Number **|** *current_available_credit* **|** *maximum_credit_limit*

The field delimiter character must be the vertical bar ("**|**") character. (This character is sometimes referred to as the "pipe" character.) This is a good choice because it will never occur in the data fields, and also makes the data file easy to browse manually with a text editor.

ALWAYS Update the "**dataFile.txt**" File

At the beginning of each transaction, the program checks if the "**dataFile.txt**" file exists. If it does exist, then its previous contents are "archived" for safe keeping. If the "**dataFile.txt**" file does **not** exist, then initially there are no accounts present in memory. (That is, the data in memory is empty). If the data file *does* exist, then **all** of the pre-exiting account data must be read into memory (before processing the transaction). At the end of each transaction, all of the data in memory must be written out to the data file. This way, the contents of the "**dataFile.txt**" file *always* contains the latest version of all credit card account data, even if the program has not been re-started. (Even though this practice is obviously inefficient, the benefit of always having the latest data present in the "**dataFile.txt**" file is considered worth it.)

Processing

The user enters text commands from the keyboard. At the beginning of each user transaction, all previous data is loaded into memory.The user transaction either creates a new data record (in memory) or modifies a previously existing data record. After the transaction has been completed, then all data must be written out to the data file. This requirement can pose a challenge during the debug process: if the program writes bad data to the file during one test, then that could mean the data file becomes unusable for the next test. To prevent this possibility, old data is "archived" for safe keeping. The process for accomplishing this is described in **Section 5** of this document: **Data File Archival Process**.

Transaction Types

There are two user commands ("**`create`**" and "**`verify`**") that must be supported:

- A "**`create`**" command creates a new account, and includes its data in the output file.
- A "**`verify`**" command authorizes a "purchase" or a "return" transaction. For simplicity, any VERIFY command that specifies a positive number must be considered a purchase (also known as a "debit"), and any VERIFY command that specifies a negative number must be considered a return (also known as a "credit"). A purchase (debit) reduces the available credit, while a return increases the available credit.

Transaction Acceptance or Rejection

There are several scenarios that are possible when the user executes the "**`verify`**" command. The program outputs a message to the screen, indicating which of these results has actually occurred:

- AUTHORIZATION GRANTED -- if the account exists, and the VERIFY transaction does not cause the account to exhaust the remaining credit line, then this must be reported to the user, along with the remaining available credit.
- AUTHORIZATION DENIED -- if the account exists, but the requested purchase would exceed the remaining available credit, then the purchase must be rejected. Also, a "credit" transaction must **not** be allowed to increase the available credit above the maximum.
- ACCOUNT NOT ON FILE – if the account is not in the data file, then obviously, no purchase or return can be processed.

2. Initial Sample Input / Output

Test your program with different situations, both valid and invalid. The sample output that follows shows correct results for several test cases. Your error messages must be appropriate for whatever error your program detects, but your error messages do ***not*** need to be identical to the examples shown. (In these examples, the text that the user types is shown in **BOLD** font. The actual input / output will all be displayed in the same font.)

Initial Sample Console Input / Output: session 1 (no previous data file present)

```
This program accepts the following inputs and performs the corresponding actions:
USAGE:   command parameters(s)
    COMMAND        PARAMETER(S)
    help                               Output help text.
    create         issuerSymbol        CREATE new account.
    verify         accountNum amount   VERIFY a purchase or credit.
    q                                  exit the program.

The command and issuerSymbol values are NOT case sensitive.
The following credit cards are supported:
     CARD                  SYMBOL
     American Express      AE
     Visa                  V
     MasterCard            MC
     Discover              DIS
     Diners Club           DINE

Command: create DINE
userInput=create DINE
New account created for credit card symbol DINE:  account number=7412075080374409,
credit limit=$500.00
1 record(s) written to output file.
Command: create V
userInput=create V
1 record(s) read from input file.
New account created for credit card symbol V:  account number=4120405764198727,
credit limit=$500.00
2 record(s) written to output file.
Command: create AE
userInput=create AE
2 record(s) read from input file.
New account created for credit card symbol AE:  account number=3211702766434232,
credit limit=$1000.00
3 record(s) written to output file.
```

Initial Sample Console Input / Output: session 1 (no previous data file present)

```
Command: create MC
userInput=create MC
3 record(s) read from input file.
New account created for credit card symbol MC:  account number=5063314623809552,
credit limit=$1000.00
4 record(s) written to output file.
Command: create DIS
userInput=create DIS
4 record(s) read from input file.
New account created for credit card symbol DIS:  account number=6032535975932964,
credit limit=$1000.00
5 record(s) written to output file.
Command: verify 5063314623809552 566
userInput=verify 5063314623809552 566
5 record(s) read from input file.
AUTHORIZATION GRANTED (accountNum=5063314623809552, transactionAmount=$566.00,
available credit=$434.00)
5 record(s) written to output file.
Command: verify 6032535975932964 1200
userInput=verify 6032535975932964 1200
5 record(s) read from input file.
AUTHORIZATION DENIED (accountNum=6032535975932964, transactionAmount=$1200.00,
available credit=$1000.00)
5 record(s) written to output file.
Command: verify 5063314623809552 -600
userInput=verify 5063314623809552 -600
5 record(s) read from input file.
AUTHORIZATION GRANTED with CAUTION -- current transaction attempted to raise
available credit above MAX=$1000.00 (accountNum=5063314623809552,
transactionAmount=$-600.00, NEW available credit =$1000.00)
5 record(s) written to output file.
Command: verify 3211702766434232 123.45
userInput=verify 3211702766434232 123.45
5 record(s) read from input file.
AUTHORIZATION GRANTED (accountNum=3211702766434232, transactionAmount=$123.45,
available credit=$876.55)
5 record(s) written to output file.
Command: verify 3211702766434223 200
userInput=verify 3211702766434223 200
5 record(s) read from input file.
ACCOUNT NOT ON FILE (accountNum=3211702766434223)
5 record(s) written to output file.
Command: q
userInput=q
Exit program.
```

Resulting Data File

The previous interactive session (**Initial Sample Console Input / Output: session 1**) results in this data file":

```
7412075080374409| 500.00| 500.00
4120405764198727| 500.00| 500.00
3211702766434232| 876.55|1000.00
5063314623809552|1000.00|1000.00
6032535975932964|1000.00|1000.00
```

3. More Sample Input / Output

Now that we have a **`dataFile.txt`** file that contains real data, suppose the user starts the program again:

More Sample Input / Output session: session 2

```
This program accepts the following inputs and performs the corresponding actions:
USAGE:   command parameters(s)
    COMMAND        PARAMETER(S)
    help                                Output this help text.
    create         issuerSymbol         CREATE new account.
    verify         accountNum amount    VERIFY a purchase or credit.
    q             exit the program.

The command and issuerSymbol values are NOT case sensitive.
The following credit cards are supported:

     CARD                   SYMBOL
     American Express       AE
     Visa                   V
     MasterCard             MC
     Discover               DIS
     Diners Club            DINE

Command: create V
userInput=create V
5 record(s) read from input file.
New account created for credit card symbol V:  account number=4964546767740262,
credit limit=$1000.00
6 record(s) written to output file.
Command: create V
userInput=create V
6 record(s) read from input file.
New account created for credit card symbol V:  account number=4932952663619606,
credit limit=$500.00
7 record(s) written to output file.
Command: create V
userInput=create V
7 record(s) read from input file.
New account created for credit card symbol V:  account number=4692558148511104,
credit limit=$1000.00
8 record(s) written to output file.
Command: create MC
userInput=create MC
8 record(s) read from input file.
```

More Sample Input / Output session: session 2

```
New account created for credit card symbol MC:  account number=5396383494216655,
credit limit=$500.00
9 record(s) written to output file.
Command: verify 5396383494216655 499
userInput=verify 5396383494216655 499
9 record(s) read from input file.
AUTHORIZATION GRANTED (accountNum=5396383494216655, transactionAmount=$499.00,
available credit=$ 1.00)
9 record(s) written to output file.
Command: q
userInput=q
Exit program.
```

Final Data File

The interactive session shown on the previous pages (**Sample Console Input / Output: session 2**) results in the data file shown below:

```
7412075080374409| 500.00| 500.00
4120405764198727| 500.00| 500.00
3211702766434232| 876.55|1000.00
5063314623809552|1000.00|1000.00
6032535975932964|1000.00|1000.00
4964546767740262|1000.00|1000.00
4932952663619606| 500.00| 500.00
4692558148511104|1000.00|1000.00
5396383494216655|   1.00| 500.00
```

4. Design Suggestion: Two Classes

The discussion below describes how I implemented this solution. Please consider this a *suggestion* to help you think about the problem in an object oriented way, but ***not*** an absolute requirement. (If you have a different design in mind, feel free to implement that. However, you should think through your design carefully, to be sure it will meet all of the project requirements.) The two classes in my implementation are called "**Credit**" and "**CreditAccount**".

The "Credit" Class

The "**Credit**" class is the class that includes the "main" method. It reads the input commands from the console and processes the two major user commands, CREATE and VERIFY. The "main" method creates a local variable which is an array of "CreditAccount" objects. This is the most critical feature of the design: the program does NOT try to modify the data file directly, but instead reads the entire input data file into memory (into the array of **CreditAccount** objects). The user transaction is processed **entirely** in memory, and then the entire contents of the **CreditAccount** array are written out to a new version of the output file. This ***read-process-write*** approach is followed for each user command.

In addition to the "main" method, the **Credit** class includes these methods:

- **loadData**: loads data from the input file.
- **writeData**: writes data to the output file.
- **createAccount**: creates a new account in memory.
- **authorizeTransaction**: verifies a transaction and modifies the "current available credit" amount.

These major methods call other "utility" methods:

- **findAccount**: searches the data (in memory) for the account number specified by the user.
- **renameWithTimestamp**: renames the existing data file (`dataFile.txt`) with a timestamp in the file name.

The "CreditAccount" Class

The second class, called "**CreditAccount**" is where most of the detailed processing takes place. Some of its data members are described in the table below.

Private Data Member	Description
`private String accountNum;`	Credit Card account number
`private double available;`	Current available credit
`private double maxLimit;`	Maximum credit for the account. (At all times, the `available` amount must be less than or equal to the `maxLimit` amount.)
`private String issuerSymbol;`	Credit Card Issuer Symbol
`private boolean accountValid;`	Flag to identify a "valid" vs. "invalid" account.

The program defines several constant values associated with the "**CreditAccount**" class:

Credit Card issuer Symbols (constants)	Description
`public static final String ISSUER_AMER_EXPRESS = "AE";`	American Express
`public static final String ISSUER_VISA = "V";`	Visa
`public static final String ISSUER_MASTER_CARD = "MC";`	MasterCard
`public static final String ISSUER_DISCOVER = "DIS";`	Discover
`public static final String ISSUER_DINERS_CLUB = "DINE";`	Diners Club

Credit Card issuer Codes – first digit of account number (constants)	Description
`public static final int ISSUER_CODE_AE = 3;`	American Express
`public static final int ISSUER_CODE_V = 4;`	Visa
`public static final int ISSUER_CODE_MC = 5;`	MasterCard
`public static final int ISSUER_CODE_DIS = 6;`	Discover
`public static final int ISSUER_CODE_DINE = 7;`	Diners Club

In addition to the normal "getters" and "setters", some of the methods in the **CreditAccount** class are described below.

Method Name, parameters	Description
`public CreditAccount(String issuerSymbol)`	Constructor for creating a new account
`public CreditAccount(String recordText, int recordLength)`	Constructor for loading records from the data file

Method Name, parameters	Description
`public String assembleRecordText()`	Create a string in the format suitable for writing to the data file.
`public static int getIssuerCode(String issuerSymbol)`	Check issuer symbol and return corresponding issuer code
`public static String getIssuerSymbol(int issuerCode)`	Get Issuer Symbol from issuer code.

5. Data File Archival Process

The approach described in this section is the mechanism by which old data is "archived". This approach gives the programmer more control of the data file during the debug process. Specifically, if one debug session produces a "garbage" output file, then it is easy for the programmer to throw that particular file away, and revert to an earlier, hopefully correct, version of the data. The disadvantage of this approach is that it leaves lots of old data files lying around, which the programmer must eventually delete, by using an appropriate manual operating system command.

The program, as described in this document, saves the credit card data in a file named "**`dataFile.txt`**". If a pre-existing file with this same name already exists, then the program renames the preexisting file to a filename that contains the current date and time, in the following format:

`dataFile_YYYYMMDD_HHMMSS.txt`

where **YYYYMMDD** is the current year, numeric month and day, and
HHMMSS is the current time-of-day in 24-hour format.

The "timestamped" file becomes the *input* file, and a **new** file named "**`dataFile.txt`**" becomes the *output* file. The result of this approach is that, after each user command, the *most recent* program output is always contained in "**`dataFile.txt`**", but older versions are preserved. Maintenance (cleanup) of old data files is NOT automatically done, and if your program crashes before creating the output file, then you will need to manually rename the old data file to the default "**`dataFile.txt`**" name.

6. Project Deliverables:

The project source code (**`*.java`**) file(s) must be submitted to *Moodle*, using the *Moodle* Activity:
CSC239_Project2

Submit *only* your **`.java`** file(s) that you have created. I will need to compile your code on my home computer in order to grade it. If you are submitting more than one (**`.java`**) file, do **NOT** enclose the files in a ZIP file. *Moodle* will allow you to submit multiple files.

7. Grading Criteria

The project will be graded according to the following grading criteria:

Program Header Documentation

The source code (**.java**) file(s) that you submit as part of your project must have an appropriate "header" comment at the top of each file: the header must include **at least** the information stated in the example below:

```
/*
*  CSC-239 Project 2:   Credit Card Verification Application
*  Student:     yourName
*  Due Date:    duesDate
*  Description: This program processes credit card transactions,
*               saving the results in a text file.
*/
```

Names Matter

The names chosen for variables and methods must be descriptive, relevant, not too short and not too long. Do NOT just choose whatever arbitrary name pops into your head. Think about what a variable means, and what role it plays in the program; then, choose a name that helps the reader understand the code.

Neatness Counts

The program source code must be clearly commented, with relevant information clearly stated, and extraneous garbage removed. "Extraneous garbage" includes:

- **"DEBUG" statements:** while printing intermediate results is an effective debugging technique, these statements must be removed or disabled before the final version is submitted.
- **Failed attempts:** If you have sections of code that you have abandoned because you replaced them with a different approach, then these non-working sections must be removed before submitting the project for grading. It is a good idea to save a copy of the non-working code for yourself, under some other filename, but the code you submit must be "clean". (Naming suggestion: a good name for the "first draft, non-working" version of a project might be: **`project2_first_draft.txt`**. The reason to change the filename suffix from "**`.java`**" to "**`.txt`**" is to prevent the compiler from trying to compile the old version.)

Project Grading Policy

Student programs are graded on the basis of correctness of program output, and clear complete presentation of the software to a potential reader not familiar with the project.

The various categories of project evaluation are weighted in the following manner:

Correct Result(s)	70%
Program Structure: The program must be organized with appropriate methods. For example, do **not** put most of your code in the "main" method or any other method.	10%
Clear and adequate comments.	10%
Good variable names and method names.	10%
Total	**100%**

Copyright © 2025 Peter Morgan. All rights reserved. You may **not** share this document with anyone or use it in any way other than as a participant in this course.